LAB 6: CODE AND BRANCH COVERAGE ANALYSIS

Event Ticket Tier Application

Name: Rekha Dhorigol

SRN: PES1UG24CS370

Section: G

1. Objective

The objective of this lab was to measure code (line) coverage and branch coverage for the Event Ticket Tier application, expand the test cases to exercise normal & edge-case behaviour and verify the results using pytest & Python coverage.

2. Work Performed

I reviewed the supplied application & created two separate test files: CodeCoverageTest_event_ticket_tier.py for code coverage & BranchCoverageTest_event_ticket_tier.py for branch coverage. The tests cover input validation, confirmed and unconfirmed manifests, supported and unsupported ticket tiers, order quantities, discounts, fraud-risk handling, lounge add-ons, tax, audit status and gate-flow congestion conditions.

3. Code Coverage Analysis

I ran the code coverage tests with pytest and generated a coverage report. The tests executed all 57 statements in event_ticket_tier.py, with no missed statements.

Code: (CodeCoverageTest_event_ticket_tier.py)

```
import pytest
from event_ticket_tier import process_ticket_manifest, monitor_gate_flow

def manifest(tier="general", orders=None, confirmed=True, lounge=False):
    return {
        "is_confirmed": confirmed,
        "tier": tier,
        "orders": orders or [],
        "exclusive_lounge": lounge,
    }

def test_general_vip_and_premium_tiers():
    for tier, price in [("general", 45), ("premium", 90), ("vip", 150)]:
        result = process_ticket_manifest(
            [manifest(tier, [{"qty": 1}])], 0
        )
        assert result["total_revenue"] == price
```

```
        assert result["status"] == "COMPLETED"

def test_unconfirmed_and_unknown_tier():
    result = process_ticket_manifest(
        [manifest(confirmed=False), manifest("invalid")], 0
    )
    assert result["status"] == "ZERO_SALES"

def test_order_quantity_and_promotions():
    result = process_ticket_manifest(
        [manifest("general", [
            {"qty": 0},
            {"qty": 2, "promo": "EARLYBIRD"},
            {"qty": 5},
        ])],
        0.1,
    )
    assert result["total_revenue"] == 304.7
    assert result["status"] == "COMPLETED"

def test_fraud_vip_and_non_vip():
    result = process_ticket_manifest(
        [
            manifest("vip", [
                {"qty": 1, "promo": "FRAUD_RISK"},
                {"qty": 2},
            ]),
            manifest("general", [
                {"qty": 1, "promo": "FRAUD_RISK"},
                {"qty": 1},
            ]),
        ],
        0,
    )
    assert result["total_revenue"] == 90
    assert result["status"] == "COMPLETED"

def test_lounge_tax_and_zero_sales():
    result = process_ticket_manifest(
        [manifest("general", [{"qty": 1}], lounge=True)], 0.1
    )
    assert result["total_revenue"] == 132
```

```
    assert process_ticket_manifest([], 0)["status"] == "ZERO_SALES"

@pytest.mark.parametrize(
    "manifests,tax",
    [
        ("invalid", 0),
        ([], -0.1),
    ],
)
def test_invalid_input(manifests, tax):
    with pytest.raises(ValueError):
        process_ticket_manifest(manifests, tax)

def test_audit_rejection():
    orders = [{"qty": 0} for _ in range(5)]
    result = process_ticket_manifest([manifest(orders=orders)], 0)
    assert result["status"] == "AUDIT_REJECTED"

def test_gate_flow():
    assert monitor_gate_flow([]) == "NO_DATA"
    assert monitor_gate_flow([[10, 20, 30]]) == "FLOW_NORMAL"
    assert monitor_gate_flow([[51, 52]]) == "OVERFLOW"
```

```
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6> python -m coverage erase
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6> python -m coverage run --source=event_ticket_tier -m pytest -v CodeCoverageTes
t_event_ticket_tier.py
==================== test session starts ====================
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0 -- C:\Program Files\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6
plugins: anyio-4.11.0
collected 9 items

CodeCoverageTest_event_ticket_tier.py::test_general_vip_and_premium_tiers PASSED [ 11%]
CodeCoverageTest_event_ticket_tier.py::test_unconfirmed_and_unknown_tier PASSED [ 22%]
CodeCoverageTest_event_ticket_tier.py::test_order_quantity_and_promotions PASSED [ 33%]
CodeCoverageTest_event_ticket_tier.py::test_fraud_vip_and_non_vip PASSED [ 44%]
CodeCoverageTest_event_ticket_tier.py::test_lounge_tax_and_zero_sales PASSED [ 55%]
CodeCoverageTest_event_ticket_tier.py::test_invalid_input[invalid-0] PASSED [ 66%]
CodeCoverageTest_event_ticket_tier.py::test_invalid_input[manifests1--0.1] PASSED [ 77%]
CodeCoverageTest_event_ticket_tier.py::test_audit_rejection PASSED [ 88%]
CodeCoverageTest_event_ticket_tier.py::test_gate_flow PASSED [100%]

==================== 9 passed in 0.11s ====================
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6> python -m coverage report -m
Name                    Stmts   Miss  Cover   Missing
-----------------------------------------------------
event_ticket_tier.py       57      0   100%
-----------------------------------------------------
TOTAL                      57      0   100%
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6>
```

CODE COVERAGE TEST RESULTS & COVERAGE REPORT

Result: 100% code coverage (57 statements, 0 missed).

4. Branch Coverage Analysis

I ran the branch coverage suite with branch tracking enabled. The tests exercise both outcomes of decision points and important alternative paths, including validation failures, ticket-tier selection, quantity and promotion conditions, fraud handling, audit thresholds, and gate-flow congestion logic.

Command used: python -m coverage run --branch --source=event_ticket_tier -m pytest -v BranchCoverageTest_event_ticket_tier.py

Code: (BranchCoverageTest_event_ticket_tier.py)

```
import pytest
from event_ticket_tier import process_ticket_manifest, monitor_gate_flow

def make_manifest(
    tier="general",
    orders=None,
    confirmed=True,
    lounge=False,
):
    return {
        "is_confirmed": confirmed,
        "tier": tier,
        "orders": [] if orders is None else orders,
        "exclusive_lounge": lounge,
    }

@pytest.mark.parametrize(
    "manifests,tax",
    [
        ("not a list", 0),
        ([], -0.01),
    ],
)
def test_validation_rejection(manifests, tax):
    with pytest.raises(ValueError):
        process_ticket_manifest(manifests, tax)

def test_unconfirmed_manifest_is_skipped():
    assert process_ticket_manifest(
        [make_manifest(confirmed=False)], 0
    )["status"] == "ZERO_SALES"

@pytest.mark.parametrize(
    "tier,expected",
    [("vip", 150), ("premium", 90), ("general", 45)],
```

```
)
def test_each_supported_tier(tier, expected):
    result = process_ticket_manifest(
        [make_manifest(tier, [{"qty": 1}])], 0
    )
    assert result["total_revenue"] == expected

def test_unknown_tier_is_rejected():
    result = process_ticket_manifest(
        [make_manifest("student")], 0
    )
    assert result["status"] == "ZERO_SALES"

def test_zero_and_negative_quantities():
    result = process_ticket_manifest(
        [make_manifest(orders=[{"qty": 0}, {"qty": -2}])], 0
    )
    assert result["status"] == "ZERO_SALES"

def test_earlybird_and_volume_discount():
    result = process_ticket_manifest(
        [make_manifest(orders=[
            {"qty": 1, "promo": "EARLYBIRD"},
            {"qty": 4},
            {"qty": 5},
        ])],
        0,
    )
    assert result["total_revenue"] == 45 * 0.8 + 180 + 205

def test_fraud_for_vip_breaks_order_loop():
    result = process_ticket_manifest(
        [make_manifest("vip", [
            {"qty": 1, "promo": "FRAUD_RISK"},
            {"qty": 1},
        ])],
        0,
    )
    assert result["total_revenue"] == 0
    assert result["status"] == "ZERO_SALES"

def test_fraud_for_general_continues_order_loop():
```

```
    result = process_ticket_manifest(
        [make_manifest("general", [
            {"qty": 1, "promo": "FRAUD_RISK"},
            {"qty": 1},
        ])],
        0,
    )
    assert result["total_revenue"] == 90

def test_lounge_addon():
    result = process_ticket_manifest(
        [make_manifest(orders=[{"qty": 1}], lounge=True)], 0
    )
    assert result["total_revenue"] == 120

def test_audit_threshold():
    orders = [{"qty": 0} for _ in range(5)]
    assert process_ticket_manifest(
        [make_manifest(orders=orders)], 0
    )["status"] == "AUDIT_REJECTED"

    orders = [{"qty": 0} for _ in range(4)]
    assert process_ticket_manifest(
        [make_manifest(orders=orders)], 0
    )["status"] == "ZERO_SALES"

def test_completed_status_and_tax():
    result = process_ticket_manifest(
        [make_manifest(orders=[{"qty": 2}])], 0.1
    )
    assert result == {
        "total_revenue": 99.0,
        "status": "COMPLETED",
    }

def test_empty_gate_data():
    assert monitor_gate_flow([]) == "NO_DATA"

def test_congestion_needs_two_consecutive_high_rates():
    assert monitor_gate_flow([[51]]) == "FLOW_NORMAL"
    assert monitor_gate_flow([[51, 52]]) == "OVERFLOW"
```

```
def test_low_rate_resets_congestion_streak():
    assert monitor_gate_flow([[51, 20, 52]]) == "FLOW_NORMAL"

def test_multiple_streams():
    assert monitor_gate_flow([[51], [52, 53]]) == "OVERFLOW"
    assert monitor_gate_flow([[], [10, 20]]) == "FLOW_NORMAL"
```

```
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6> python -m coverage erase
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6> python -m coverage run --branch --source=event_ticket_tier -m pytest -v BranchCoverageTes
t_event_ticket_tier.py
==================== test session starts ====================
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0 -- C:\Program Files\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6
plugins: anyio-4.11.0
collected 18 items

BranchCoverageTest_event_ticket_tier.py::test_validation_rejection[not a list-0] PASSED [  5%]
BranchCoverageTest_event_ticket_tier.py::test_validation_rejection[manifests1--0.01] PASSED [ 11%]
BranchCoverageTest_event_ticket_tier.py::test_unconfirmed_manifest_is_skipped PASSED [ 16%]
BranchCoverageTest_event_ticket_tier.py::test_each_supported_tier[vip-150] PASSED [ 22%]
BranchCoverageTest_event_ticket_tier.py::test_each_supported_tier[premium-90] PASSED [ 27%]
BranchCoverageTest_event_ticket_tier.py::test_each_supported_tier[general-45] PASSED [ 33%]
BranchCoverageTest_event_ticket_tier.py::test_unknown_tier_is_rejected PASSED [ 38%]
BranchCoverageTest_event_ticket_tier.py::test_zero_and_negative_quantities PASSED [ 44%]
BranchCoverageTest_event_ticket_tier.py::test_earlybird_and_volume_discount PASSED [ 50%]
BranchCoverageTest_event_ticket_tier.py::test_fraud_for_vip_breaks_order_loop PASSED [ 55%]
BranchCoverageTest_event_ticket_tier.py::test_fraud_for_general_continues_order_loop PASSED [ 61%]
BranchCoverageTest_event_ticket_tier.py::test_lounge_addon PASSED [ 66%]
BranchCoverageTest_event_ticket_tier.py::test_audit_threshold PASSED [ 72%]
BranchCoverageTest_event_ticket_tier.py::test_completed_status_and_tax PASSED [ 77%]
BranchCoverageTest_event_ticket_tier.py::test_empty_gate_data PASSED [ 83%]
BranchCoverageTest_event_ticket_tier.py::test_congestion_needs_two_consecutive_high_rates PASSED [ 88%]
BranchCoverageTest_event_ticket_tier.py::test_low_rate_resets_congestion_streak PASSED [ 94%]
BranchCoverageTest_event_ticket_tier.py::test_multiple_streams PASSED [100%]

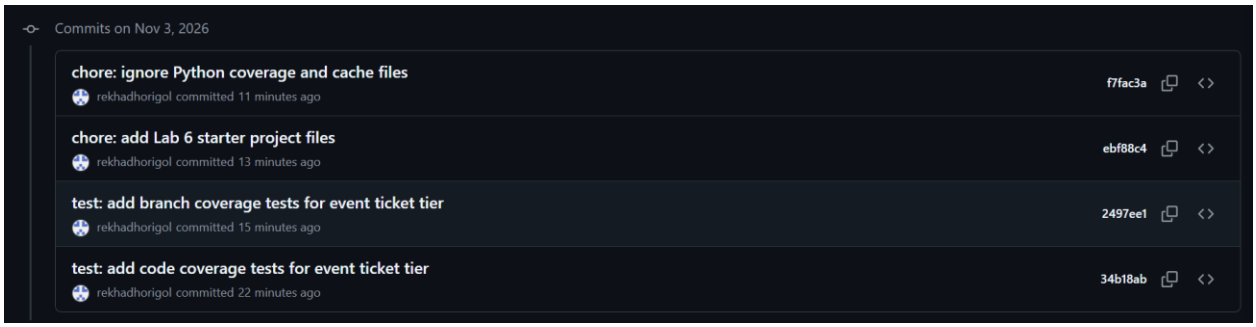
==================== 18 passed in 0.16s =====================
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6> python -m coverage report -m
Name                  Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
event_ticket_tier.py      57      0     40      0   100%
-----------------------------------------------------------------
TOTAL                     57      0     40      0   100%
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-6>
```

BRANCH COVERAGE TEST RESULTS & COVERAGE REPORT

Result: 18 tests passed. Branch coverage was 100% (57 statements, 0 missed; 40 branches, 0 partial branches).

5. Git Commit Evidence

The code coverage test and its screenshot, and the branch coverage test and its screenshot, were committed separately as required.



GIT COMMIT HISTORY INCLUDING THE TWO COVERAGE COMMITS

6. Conclusion

The test suite achieved 100% code coverage and 100% branch coverage for the supplied application, exceeding the minimum requirement of 90%. The two test files and their terminal-output screenshots are available in the repository.

GitHub repository: https://github.com/rekhadhorigol/SE-Labs-PES1UG24CS370